

Character Map for Efficient Partition Pruning via Absence Detection

Mengdi Liao (menl@itu.dk)

May 14, 2026

Contents

1	Introduction	2
1.1	Topic	2
1.2	Motivation	2
1.3	Research Question	3
1.4	Contributions	3
2	Design of the Character Map	4
2.1	Data Representation	4
2.2	Query-Time Pruning	6
3	Experiment	7
3.1	Design	7
3.2	Results	10
3.2.1	Partition-Size Sweep	10
3.2.2	Fixed-Partition $L_{\max}$ Sweep	11
3.2.3	Trade-offs	11
3.2.4	Conclusion	13
4	Related Work	13
4.1	Column Imprints	14
4.2	Cuckoo Index	14
5	Summary	15
5.1	Findings	15
5.2	Future Work	15
6	Declaration of Generative AI Use	16

1 Introduction

Efficient data pruning is a critical component in large-scale data processing systems, where reducing unnecessary data access directly impacts query performance. A commonly used approach is min-max-based pruning, which stores the minimum and maximum values of each partition and eliminates partitions whose ranges cannot contain the query value. However, this approach relies heavily on the values within a partition forming a selective range. When the data is not well ordered by the query key, or when each partition contains widely dispersed values, the min-max range can become too broad to prune effectively. This work investigates whether a bitmap-inspired character map can provide an alternative partition summary: instead of representing a range, it captures position-wise character presence and uses this information to rule out partitions that cannot contain a query.

1.1 Topic

This work is situated in the domain of data pruning and query optimization in large-scale data processing systems.

At a high level, data pruning techniques aim to reduce the search space by eliminating irrelevant data before full query evaluation, thereby improving overall system efficiency.

Within this domain, this study focuses on partition-level filtering, where summaries are used to determine whether a data partition can be safely skipped.

In particular, the work explores absence-based pruning using a bitmap-inspired character map representation, as an alternative to traditional min-max range summaries.

1.2 Motivation

In large-scale data processing, the cost of query execution is often dominated by the amount of data that must be scanned. Partitioning alone does not reduce data access unless the system can determine which partitions are irrelevant to a query. This determination is typically supported by partition bounds, constraints, or lightweight partition-level summaries such as min-max metadata[1]. When these summaries are too coarse, many partitions may still need to be accessed, increasing query latency and resource consumption. As datasets continue to grow, improving pruning efficiency becomes increasingly important for maintaining system performance.

Min-max-based pruning(also known as zone maps) is widely adopted due to its simplicity and low overhead. A simple example is shown below:

However, its effectiveness is closely tied to data distribution. In scenarios where value ranges overlap across partitions, min-max summaries provide limited discrimination, causing many irrelevant partitions to remain candidates for evaluation. This is particularly limiting when the query asks whether a specific value may exist in a parti-

Query: value = 50			
Partition	Min	Max	Decision
P1	1	30	→ Prune
P2	40	60	→ Keep
P3	70	100	→ Prune

Table 1: Example of min-max pruning for the query value 50. Partitions are kept only when the query value falls within their stored min-max range.

tion. We refer to this setting as absence-based pruning: the goal is to use the summary to prove that a partition cannot contain the queried value. Because min-max summaries only store range boundaries, they lack the feature-level information needed to rule out many absent values.

These limitations suggest that there is potential for alternative summary representations that capture richer information while remaining computationally efficient. In particular, leveraging feature-level encoding may enable more precise pruning decisions and reduce unnecessary data access.

1.3 Research Question

This study investigates whether a bitmap-inspired character map can improve pruning effectiveness compared to the widely used min-max approach. The evaluation focuses on different data types and partition configurations, while also examining the impact of the tracked string length on the behavior of the proposed method.

Main research question The research aims to answer the following question:

To what extent can a character map-based approach improve partition pruning effectiveness over min-max methods?

Future extensibility This study does not further investigate larger character domains, but the fixed 64-symbol alphabet highlights an important direction for future work. Later studies could examine how the character map design should be adapted to support larger alphabets, multilingual text, or Unicode characters.

1.4 Contributions

The main contributions of this work are:

- A bitmap-inspired character map representation for partition-level pruning, constructed during a precomputation stage to encode feature presence within each partition.
- A query-time pruning mechanism that leverages bitwise operations on the character map to enable efficient and deterministic absence-based filtering.

- An experimental evaluation comparing the proposed approach with min–max–based pruning across different data types and partition sizes, and exploring the behavior of the character map under varying configurations.
- An analytical study of the proposed method, providing insights into its performance characteristics, trade-offs, and limitations.

2 Design of the Character Map

The Character Map (Charmap) is a bitmap-based summary structure for efficient absence checking. It represents the presence of characters in a data partition using a fixed-length bit vector, where each bit corresponds to a character in the domain. Charmap operates in two phases: a precomputation phase that builds a bitmap summary per partition, and a query phase that converts queries into the same representation for comparison.

2.1 Data Representation

We represent a collection of strings using a *position-aware character map*. Each character position in a string is associated with a fixed-size bit vector, and the full representation consists of an array of bitsets:

$$\text{CharSet}[0], \text{CharSet}[1], \dots, \text{CharSet}[L_{\max} - 1]$$

Each $\text{CharSet}[i]$ is a 64-bit vector that summarizes the characters observed at string position i across all strings in a partition. A bit is set to 1 if at least one string in the partition contains the corresponding character at that position.

Maximum Tracked Positions $L_{\max}$

The parameter $L_{\max}$ denotes the maximum number of string positions tracked by the representation. It defines the number of positional bitsets maintained in the character map. For any string s with length $|s| < L_{\max}$, only the first $|s|$ positions contribute to the character map. In this project, $L_{\max}$ is treated as a predefined upper bound. The choice of $L_{\max}$ introduces a trade-off between space and precision: a larger value captures more positional information, but also increases the size of each partition summary.

Alphabet Definition

The alphabet is fixed and consists of the following 64 symbols:

- Uppercase letters: A–Z (26)
- Lowercase letters: a–z (26)

- Digits: 0–9 (10)
- Special characters: `_` and `-` (2)

Each character is deterministically mapped to an integer index in the range $[0, 63]$, which corresponds to a bit position within the 64-bit vector. This bit position is different from the character’s position within the string: `CharSet[i]` selects the string position, while the bit index inside `CharSet[i]` identifies the character.

For readability, examples often show the observed character set directly, such as $\{t, r, p\}$. Internally, this set is stored as a 64-bit vector. For example, if only the lower-case positions around `p`, `r`, and `t` are shown, the set $\{p, r, t\}$ is encoded by setting the corresponding bits to 1:

Character	<i>p</i>	<i>q</i>	<i>r</i>	<i>s</i>	<i>t</i>
Bit value	1	0	1	0	1

All other character positions in the 64-bit vector are set according to whether their corresponding characters appear in the same string position.

The total memory footprint per partition is therefore:

$$64 \times L_{\max} \text{ bits}$$

Example

Figure 1 illustrates the construction of a Character Map for a small partition. The columns denote character positions within the strings. For each position, the observed characters are aggregated across all strings in the partition and encoded using the 64-symbol alphabet mapping described above.

String	Char at $i = 0$	Char at $i = 1$	Char at $i = 2$
cat	c	a	t
car	c	a	r
cap	c	a	p
Aggregated set	{c}	{a}	{t,r,p}
Character map	CharSet[0]	CharSet[1]	CharSet[2]

Figure 1: Position-wise Character Map construction. Characters are aggregated by position across all strings in a partition, producing one bitset per tracked position.

Multiple distinct strings may produce identical character maps. As a result, the representation can safely identify some non-matching cases, but it may also produce false positives that require further verification.

2.2 Query-Time Pruning

After the character maps have been precomputed, each query string is evaluated against the summary of every partition. Given a query string q , the evaluation process examines the characters of q position by position, up to $\min(|q|, L_{\max})$. For each position i , the character $q[i]$ is mapped to its corresponding bit position and checked against $\text{CharSet}[i]$ of the partition.

Pruning Condition

A partition can be safely pruned if any character in the query is absent from the corresponding position in the character map. Formally, this condition is defined as:

$$\exists i < \min(|q|, L_{\max}) : \text{bit}(q[i]) \notin \text{CharSet}[i]$$

In this case, no string in the partition contains the same character as the query at position i , and therefore the partition cannot contain the query string. The partition is classified as a definite non-match.

Candidate Retention

If all tested positions satisfy the bitmap condition, the partition is retained as a candidate:

$$\forall i < \min(|q|, L_{\max}) : \text{bit}(q[i]) \in \text{CharSet}[i]$$

This result indicates that the partition may contain the query, but it does not guarantee an exact match. The character map stores positional character sets independently rather than complete strings, which means dependencies between positions are not preserved.

Example

Consider a partition containing the strings:

cat
dog

The resulting character map contains $\{c, d\}$ at position 0, $\{a, o\}$ at position 1, and $\{t, g\}$ at position 2. Table 2 illustrates the position-wise matching process for the query `cog`.

Although all positions satisfy the bitmap condition, the string `cog` is not actually present in the partition. This example demonstrates a false positive: the summary may retain a partition even when the exact query string is absent.

However, pruning is safe when a bit is missing. If $q[i]$ is not contained in $\text{CharSet}[i]$, then no string in the partition has the same character at position i , so the partition

Position	CharSet	Query Character	Match
0	{c, d}	c	✓
1	{a, o}	o	✓
2	{t, g}	g	✓

Table 2: Position-wise evaluation of query `cog`.

cannot contain q . Thus, the character map may produce false positives, but missing-bit pruning does not introduce false negatives.

3 Experiment

3.1 Design

The goal of the experiment is to evaluate how effectively the Character Map summary can prune irrelevant partitions during query processing. We compare it against a MinMax baseline under the same partitioning scheme and query workload.

Datasets and Preprocessing

Dataset split The experiments are conducted on two types of string data: IP addresses and personal names. The IP address data is obtained from [2], and the name dataset is obtained from [3]. For each dataset, we use one million records to build the partition summaries and 1,000 records as the query workload.

The query workloads are generated differently for the two datasets. For the IP dataset, 800 query addresses are sampled from the build data, while 200 query addresses are newly generated random addresses. This produces a mixture of queries that may be present in the build set and queries that are expected to be absent. For the name dataset, the 1,000 query records are randomly sampled from the name collection using the general row-sampling script.

Dataset	Build Records	Query Records	Query Composition
IP addresses	1,000,000	1,000	80% build, 20% random
Names	1,000,000	1,000	Randomly sampled from name collection

Table 3: Dataset split used in the experiments

IP preprocessing. The IP dataset contains many entries in CIDR notation. Since the Character Map operates on strings rather than network prefixes, each IP address is first normalized by removing any suffix after the slash character. For example, `192.168.1.0/24` is treated as `192.168.1.0`. The four octets are then converted into a fixed-width 12-digit string by padding each octet with leading zeros:

1.2.3.4 $\rightarrow$ 001002003004

This normalization removes separator characters from the representation and pads each octet to the same width, ensuring that the same string positions correspond to the same octet digits across all IP addresses.

Name preprocessing. For the name dataset, we only keep names whose characters are supported by the Character Map alphabet: uppercase letters, lowercase letters, digits, underscore, and hyphen. During the experiment, only the first name is considered; last names and any remaining parts of the line are ignored. The original casing is preserved, so uppercase and lowercase characters are treated as distinct symbols.

Precomputed summary storage. Both build records and query records are processed using the same normalization rules. Before running the query experiments, the partition summaries are precomputed and stored on disk.

Internally, the Character Map summary is accumulated using bit operations: each character is mapped to one of 64 bit positions, and each position-wise character set is stored as a 64-bit bitmap. However, the precomputed output files are written in a human-readable textual format. Instead of storing raw bit vectors as sequences of zeros and ones, the implementation converts each bitmap back into the corresponding set of observed characters. Therefore, a row in a precomputed file shows the characters observed at each position for a partition, rather than the underlying binary representation.

Experimental Variables

The experiments are configured using the following variables: partition size P , maximum tracked string length $L_{\max}$, dataset type, and summary strategy. These variables define each experimental run.

Partition size P . The partition size P specifies the number of records included in each partition summary. In the partition-size sweep, we use:

$$P \in \{5, 10, 50, 100, 200, 500, 1000, 2000\}$$

For each value of P , the build dataset is divided sequentially into partitions of size P . The final partition may contain fewer than P records if the total number of records is not divisible by P .

Maximum tracked length $L_{\max}$. The parameter $L_{\max}$ specifies the number of character positions stored in a Character Map summary. Unless otherwise stated, the experiments use:

$$L_{\max} = 12$$

In the fixed-partition experiments, we also vary $L_{\max}$ using:

$$L_{\max} \in \{3, 5, 8, 12\}$$

These runs are performed for fixed values of P .

Dataset type. Each configuration is evaluated on both datasets:

- IP address dataset
- Name dataset

The same build-query split described above is used for both datasets.

Summary strategy. Each configuration is evaluated using two summary strategies:

- Character Map
- MinMax baseline

Both strategies use the same preprocessed input data, the same partition boundaries, and the same query workload.

Variable	Experimental setting
Partition size P	5, 10, 50, 100, 200, 500, 1000, 2000
Maximum tracked length $L_{\max}$	12 by default; 3, 5, 8, 12 in fixed-partition experiments
Dataset type	IP address dataset and name dataset
Summary strategy	Character Map and MinMax baseline

Table 4: Variables used in the experiments

Evaluation Metric

The main evaluation metric is the pruning ratio. For each query, the query procedure checks all precomputed partition summaries and counts how many partitions can be pruned. A partition is considered pruned if the summary determines that the query cannot be contained in that partition.

For a single query q , the pruning ratio is defined as:

$$\text{Pruning Ratio}(q) = \frac{\text{Pruned Partitions for } q}{\text{Total Partitions}}$$

The query output file records the number of pruned partitions for each query. Each output file also stores metadata such as the source precompute file, the summary strategy,

the partition size P , the value of $L_{\max}$ when applicable, the total number of partitions, and the number of query strings.

For each experimental configuration, we report the average pruning ratio across all query strings:

$$\text{Average Pruning Ratio} = \frac{1}{|Q|} \sum_{q \in Q} \text{Pruning Ratio}(q)$$

where Q is the set of query strings. In the plots, error bars are computed from the standard deviation of the per-query pruning ratios.

3.2 Results

3.2.1 Partition-Size Sweep

The partition-size sweep evaluates all four series using $P \in \{5, 10, 50, 100, 200, 500, 1000, 2000\}$ with $L_{\max} = 12$, as shown in Figure 2.

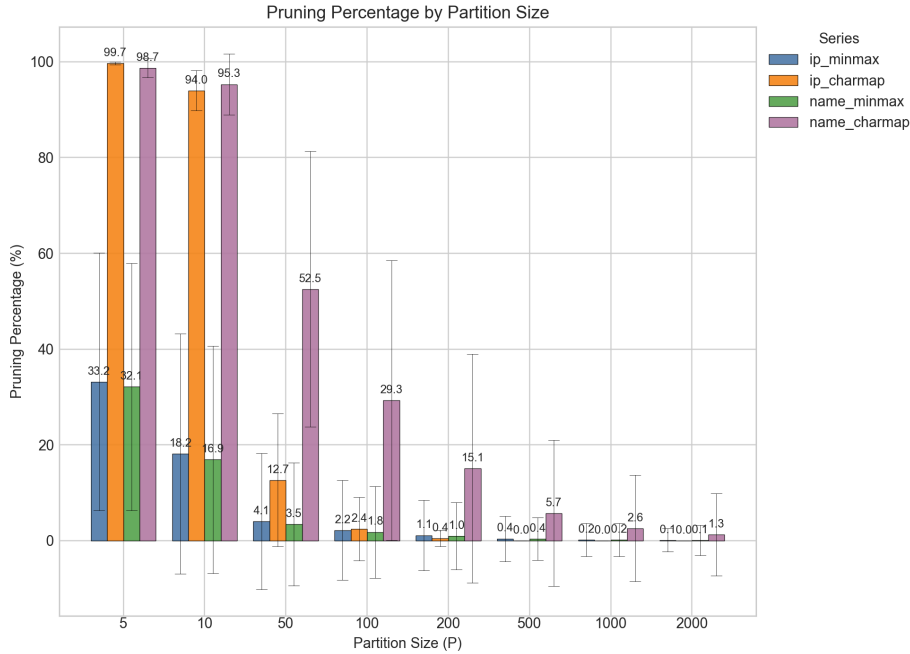


Figure 2: Pruning percentage by partition size for IP and name datasets using Character Map and MinMax summaries.

The partition-size sweep shows that pruning effectiveness decreases as the partition size increases. At small partition sizes, Charmap prunes a large fraction of partitions on both datasets. For example, when $P = 5$, Charmap achieves nearly complete pruning, with 99.7% on the IP dataset and 98.7% on the name dataset.

The gap between Charmap and MinMax is largest at small partition sizes. While Charmap remains above 94% for both datasets at $P = 10$, the MinMax baseline is

below 20%. This shows that the position-wise character sets provide much more selective partition summaries than the lexicographical range stored by MinMax in this setting.

As P increases, all methods lose pruning effectiveness, but the decrease is not identical across datasets. The IP Charmap result drops sharply after $P = 10$, reaching only 2.4% at $P = 100$. In contrast, the name Charmap retains more pruning power at medium partition sizes, with 29.3% at $P = 100$ and 15.1% at $P = 200$.

For large partition sizes, the pruning percentages of all methods approach zero. This indicates that when many records are summarized together, the partition summaries become too coarse to eliminate many partitions. Overall, the partition-size sweep shows that Charmap is most effective when partition sizes are small enough for the position-wise character sets to remain selective.

3.2.2 Fixed-Partition $L_{\max}$ Sweep

The fixed-partition experiments vary $L_{\max} \in \{3, 5, 8, 12\}$ while holding the partition size fixed. The experiments are reported for $P = 100$, $P = 200$, and $P = 500$, as shown in Figures 3a, 3b, and 3c.

In the fixed-partition experiments, $L_{\max}$ only applies to the Character Map summary. The MinMax baseline is shown as a reference, but it remains unchanged across different $L_{\max}$ values because it does not use position-wise character information. This pattern is consistent for all three fixed partition sizes.

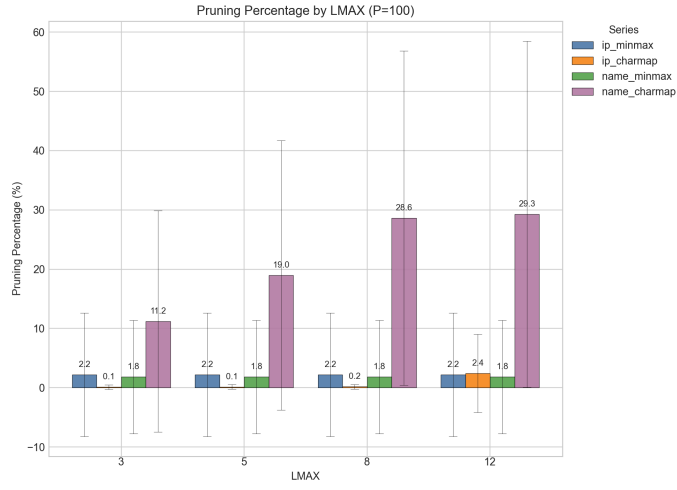
The effect of $L_{\max}$ is most visible on the name dataset. For $P = 100$, the name Character Map increases from 11.2% at $L_{\max} = 3$ to 29.3% at $L_{\max} = 12$. The same increasing pattern appears for $P = 200$ and $P = 500$, although the overall pruning percentages are lower when the partition size is larger.

The IP dataset shows a much smaller response to increasing $L_{\max}$. For $P = 100$, the IP Character Map remains near zero for $L_{\max} = 3, 5, 8$ and reaches 2.4% at $L_{\max} = 12$. For $P = 200$ and $P = 500$, the IP Character Map remains close to zero across the tested $L_{\max}$ values.

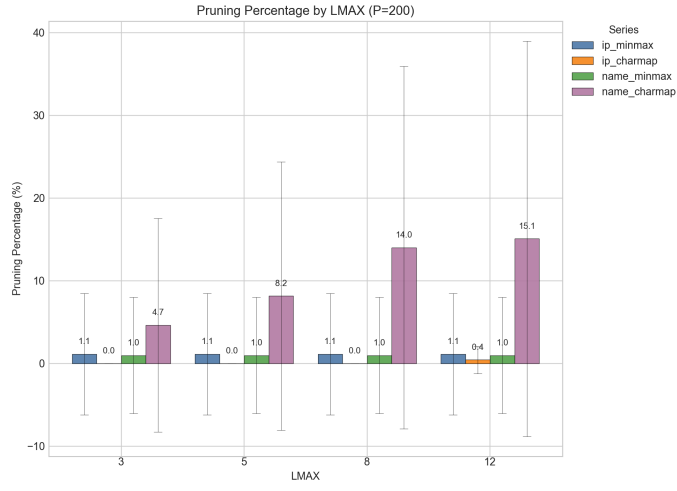
Overall, the fixed-partition sweep shows that increasing $L_{\max}$ can improve Charmap pruning when the additional positions provide useful filtering information. In these experiments, this effect is much clearer for the name dataset than for the IP dataset.

3.2.3 Trade-offs

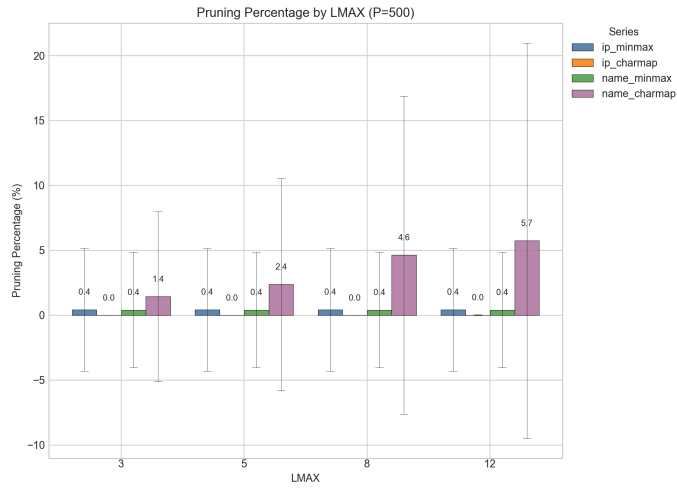
Partition size. The partition size P controls the granularity of the summaries. A smaller partition size produces more partitions, with fewer records summarized in each one. This keeps each Character Map more specific and can improve pruning effectiveness. However, it also increases the number of summaries that must be stored and checked during query processing. A larger partition size reduces the number of summaries, but each summary aggregates more records, which can reduce its selectivity.



(a) $P = 100$



(b) $P = 200$



(c) $P = 500$

Figure 3: Pruning percentage by $L_{\max}$ under different fixed partition sizes.

Maximum tracked length. The parameter $L_{\max}$ controls how many character positions are stored in each Character Map. Increasing $L_{\max}$ adds more positional bitsets and allows the query procedure to check more characters. The space cost per partition is:

$$64 \times L_{\max} \text{ bits}$$

Thus, increasing $L_{\max}$ increases the storage cost linearly. The benefit of a larger $L_{\max}$ depends on whether the additional positions provide useful distinguishing information for the workload.

3.2.4 Conclusion

The experiments show that Charmap can be used as an effective partition-level summary for absence checking. Its effectiveness is strongest when the position-wise character sets remain selective, which is most visible at smaller partition sizes. In these settings, the summary is able to eliminate many partitions before any exact verification step is needed.

Compared with the MinMax baseline, Charmap stores more detailed information about each partition. MinMax represents a partition using only its lexicographical lower and upper bounds, while Charmap stores a set of observed characters for each tracked position. This additional positional information gives Charmap more opportunities to reject non-matching partitions, but it also makes the summary larger and dependent on the chosen value of $L_{\max}$.

The results also show that Charmap is workload-dependent. The IP and name datasets exhibit different pruning behavior under the same parameter settings, which indicates that the usefulness of the position-wise bitsets depends on the underlying character distribution. If many distinct characters appear at the same positions within a partition, the bitsets become less selective and the pruning benefit decreases.

Overall, Charmap is best understood as a pre-filter rather than an exact lookup structure. It can safely identify partitions that cannot contain a query, but retained partitions still require further verification if exact membership is needed. In practice, the choice of P and $L_{\max}$ should be made according to the workload and the desired balance between summary size and pruning effectiveness.

4 Related Work

Lightweight indexing techniques are commonly used to reduce unnecessary data scans in analytical workloads through compact metadata structures. Both Column Imprints and Cuckoo Index aim to improve query efficiency by pruning irrelevant data partitions or blocks before full evaluation, while maintaining low storage overhead.

4.1 Column Imprints

Column Imprints were proposed by Sidiropoulos and Kersten as a lightweight secondary index for column-oriented database systems [4]. The method is designed to reduce unnecessary data access during predicate evaluation, especially in main-memory settings where memory traffic and cache efficiency are important.

A column imprint divides a column into small physical units, such as cache lines, and assigns one bit vector to each unit. The value domain is divided into bins, and each bit in the imprint vector corresponds to one bin. If at least one value in the unit falls into a bin, the corresponding bit is set. During query evaluation, the query predicate is mapped to the relevant bins and compared with the imprint vector using bitwise operations. If there is no overlap, the unit can be skipped; otherwise, the original values must still be checked.

Block	Values in block	Imprint bits over bins 1, ..., 8
B_1	{1, 8, 4}	1 0 0 1 0 0 0 1
B_2	{1, 6, 7}	1 0 0 0 0 1 1 0
B_3	{2, 5, 8}	0 1 0 0 1 0 0 1

Figure 4: Simplified construction of Column Imprints. Each block of column values is summarized by a bit vector, where a bit is set if at least one value in the block falls into the corresponding bin.

Column Imprints mainly report query-time speedup rather than pruning percentage. For high-selectivity range queries, the authors report up to nearly 1000× improvement over sequential scans and up to around 100× improvement over zonemaps, with storage overhead of at most about 12% over the base column.

4.2 Cuckoo Index

Cuckoo Index was proposed by Kipf et al. as a lightweight secondary index for data skipping in analytical systems [5]. It targets equality predicates in read-only settings and is designed to represent the relationship between keys and data partitions in a compact way.

The main idea is to store each distinct key only once. For each key, the index stores a fingerprint in a Cuckoo-filter-like structure, together with a bitmap indicating which partitions contain that key. During query processing, the query key is mapped to its fingerprint. If the fingerprint is found, the associated bitmap identifies the candidate partitions that need to be checked. If no matching fingerprint is found, the partitions can be skipped.

This structure differs from maintaining one filter per partition. Instead of testing every partition independently, the index maps a queried key directly to a compact repre-

Key	Partitions containing the key	Partition bitmap (P_1, P_2, P_3, P_4)
A	$\{P_1, P_3, P_4\}$	1 0 1 1
B	$\{P_1, P_2\}$	1 1 0 0
C	$\{P_2\}$	0 1 0 0
D	$\{P_3, P_4\}$	0 0 1 1

Figure 5: Simplified construction of a Cuckoo Index. Each distinct key is stored through a fingerprint, and the associated bitmap records the partitions in which the key appears.

sensation of the partitions that may contain it. The resulting bitmap acts as a candidate set for further evaluation.

Cuckoo Index evaluates pruning using scan rate, defined as the ratio of false positive stripes to true negative stripes. The authors configure CI with target scan rates of 0.1% and 1%, and report that CI can use less space for the same pruning power than per-stripe filters. In a PostgreSQL experiment, CI returns on average $2.10\times$ fewer partitions than Bloom filters and achieves a $2.45\times$ query speedup.

5 Summary

5.1 Findings

The experiments show that the Character Map can improve partition pruning effectiveness over the MinMax baseline, especially when the partition size is small. By storing position-wise character information, Charnmap is able to reject partitions that cannot contain a query more precisely than a summary based only on lexicographical minimum and maximum values.

Partition size is one of the main factors affecting pruning performance. Smaller partitions produce more selective summaries, while larger partitions aggregate more records and reduce the ability to prune. The experiments also show that increasing $L_{\max}$ can improve Charnmap pruning, but the benefit depends on whether the additional tracked positions provide useful filtering information.

The results differ between the IP and name datasets, indicating that Charnmap is sensitive to workload characteristics. In particular, the effectiveness of the position-wise bitsets depends on the character distribution within each partition.

5.2 Future Work

Future work could extend the fixed 64-symbol alphabet to larger character domains, such as Unicode, multilingual text, or more complex symbols. This would require a more general mapping strategy, since a single 64-bit vector is no longer sufficient when the character domain becomes much larger.

Another direction is to make the Character Map more adaptive. Inspired by Column Imprints, future versions could explore data-dependent character grouping or position weighting, so that the summary focuses on the characters and string positions that provide the most pruning power. Similarly, the partition size P and tracked length $L_{\max}$ could be selected based on workload and data distribution rather than fixed manually.

The query structure could also be improved. Inspired by Cuckoo Index, Charmap could be reorganized as inverted bitmaps over partitions, where each character-position pair maps directly to candidate partitions. This would allow a query to combine candidate partition sets directly instead of scanning every partition summary.

Finally, Charmap could be combined with other summary structures, such as Min-Max or lightweight fingerprints, to reduce false positives while preserving efficient pruning. Future evaluations should also include system-level metrics such as storage cost, precomputation time, and query time.

6 Declaration of Generative AI Use

During the preparation of this project, I used generative AI and AI-supported technologies as supporting tools. The tools used were Anthropic Claude and OpenAI Codex.

Anthropic Claude was used to support software development tasks, including code inspection, debugging, refactoring suggestions, README preparation, and checking the consistency between the implemented experiment runners and the stored experiment results.

OpenAI Codex was used to support writing and revision tasks, including language polishing, structural feedback, explanation refinement, and improving the clarity of selected parts of the report.

The generative AI tools were used as assistance tools only. The research idea, experimental setup, implementation decisions, interpretation of results, and final submitted text were reviewed, evaluated, and accepted by me. I take full responsibility for the content of this project report and for the correctness of the submitted code and experiment results.

References

- [1] PostgreSQL Global Development Group. Postgresql documentation: Table partitioning. <https://www.postgresql.org/docs/current/ddl-partitioning.html>. Accessed: 2026-05-11.
- [2] IPinfo. Ipinfo data downloads, 2026. Accessed: 2026-04-29.
- [3] Philippe Remy. Name dataset, 2020. Accessed: 2026-04-29.

- [4] Lefteris Sidiropoulos and Martin Kersten. Column imprints: A secondary index structure. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 893–904. ACM, 2013.
- [5] Andreas Kipf, Damian Chromejko, Alexander Hall, Peter Boncz, and David G. Andersen. Cuckoo index: A lightweight secondary index structure. *Proceedings of the VLDB Endowment*, 13(13):3559–3572, 2020.